

Road Warrior Presentation Guide

Purpose of This Guide

This guide is a complete speaking and slide-building script for presenting the Road Warrior architecture kata solution. It is designed to help the presenter explain not only what the architecture is, but why these decisions were made, what trade-offs were accepted, and how the proposed solution addresses both the problem statement and the architectural quality goals.

Use this guide to either:

- create a fresh presentation deck from scratch, or
- refine the existing deck into a tighter architecture narrative.

Recommended presentation length: 12-15 minutes.

Recommended audience framing: architecture reviewers, engineering leads, and stakeholders evaluating whether the solution is coherent, scalable, and realistic.

This guide is also aligned to the kata evaluation rubric, which explicitly scores:

- Functional and non-functional requirements
- Logical architecture
- Physical architecture
- Solution strategy and requirement-to-decision mapping
- ADR summary

Rubric Alignment

If the presentation is being judged against the kata scoring sheet, the deck should make these five areas impossible to miss.

1. Functional and Non-Functional Requirements

The presentation must clearly show:

- what the system does
- what quality attributes drive the architecture
- what constraints shape the solution

In this guide, that is primarily covered by:

- Slide 2: Problem Statement and Business Need
- Slide 3: Functional Requirements
- Slide 4: Architectural Drivers and Constraints

2. Logical Architecture

The presentation must clearly explain the system's internal building blocks, responsibilities, and interactions.

In this guide, that is primarily covered by:

- Slide 6: System Context
- Slide 7: Container Architecture
- Slide 8: Reservation Ingestion and Event Flow
- Slide 9: Trip Grouping Logic and User Experience
- Slide 11: Runtime View

3. Physical Architecture

The presentation must show how the system is deployed in a realistic environment.

In this guide, that is primarily covered by:

- Slide 13: Deployment and Operations

This slide should explicitly mention managed compute, messaging, database, scaling, observability, and network/security boundary assumptions.

4. Solution Strategy and Mapping Requirements to Decisions

The presentation must trace architectural choices back to the problem statement and quality goals.

In this guide, that is primarily covered by:

- Slide 5: Chosen Architecture Style
- Slide 8: Reservation Ingestion and Event Flow
- Slide 10: Why We Chose Backend-for-Frontend
- Slide 12: Why Azure SQL Is the Operational Database
- Slide 15: Risks and Mitigations
- Slide 16: Closing Summary

This is the section reviewers usually score hardest. Do not just say what was chosen. Always say which requirement or quality attribute that choice satisfies.

5. ADR Summary

The presentation should include a concise, easy-to-scan summary of the key decisions.

In this guide, that is primarily covered by:

- Slide 14: Key ADRs and Trade-Offs

If time is tight, keep ADRs to one line each:

- Microservices for independent scaling and fault isolation
- Event-driven ingestion for near real-time updates and decoupled integration
- WebSockets or SSE for responsive client updates
- Managed cloud deployment for elasticity and reduced operational burden

- Azure SQL for strong consistency and relational trip querying
 - Trip grouping service with user override for domain accuracy and usability
-

Presentation Strategy

The strongest presentation flow for this kata is:

1. Start from the business problem, not the technology.
2. Show the functional requirements, non-functional requirements, and constraints.
3. Explain why an event-driven microservices architecture with BFF was chosen.
4. Walk through the logical architecture using context, container, and runtime views.
5. Show the physical deployment view clearly.
6. Close with requirement-to-decision mapping, ADRs, trade-offs, and risks.

The core message to reinforce throughout the presentation is:

Road Warrior is not a booking platform. It is a high-availability, near real-time trip management dashboard that consolidates reservations from existing agency systems and presents them as coherent trips across web and mobile experiences.

Slide-by-Slide Guide

Note: if you need to shorten the deck, do not cut the rubric-bearing slides first. Preserve the slides that directly support requirements, logical architecture, physical architecture, solution mapping, and ADRs.

Slide 1: Title Slide

Slide Title

Road Warrior: Architecture for a Next-Generation Trip Management Dashboard

Objective

Set context quickly and establish that the solution is architecture-driven, not feature-list driven.

On-Slide Content

- Road Warrior
- Architecture Kata Solution
- Team name
- Date

Speaker Notes

Open with a one-sentence definition of the system:

"Road Warrior is a trip management dashboard that aggregates existing travel reservations from airline, hotel, and car rental systems, groups them into trips, and surfaces them through rich web and mobile experiences."

Then clarify scope early:

"This solution is intentionally focused on managing existing reservations, not booking new travel. That distinction shaped many of our architectural decisions."

Key Message

This is a travel visibility platform, not a booking engine.

Slide 2: Problem Statement and Business Need

Slide Title

Problem We Are Solving

Objective

Translate the problem statement into a business and user problem.

On-Slide Content

- Travelers need one place to view all reservations organized by trip.
- Data must come from existing agency integration systems.
- Users also need manual reservation entry.
- Trips must move automatically from active view to Past Trips.
- Sharing and international usability are required.

Speaker Notes

Frame the problem as fragmentation and freshness:

"Today, reservation data is fragmented across multiple travel domains: flights, hotels, and car rentals. The business need is to consolidate that fragmented data into a single traveler-facing dashboard that remains current, usable across platforms, and scalable as adoption grows."

Then mention the two hard realities:

- integration must happen through existing agency systems, not direct vendor APIs
- the system must support both automatic ingestion and manual entry

Key Message

The hardest part is not displaying trips. The hardest part is consolidating and maintaining trustworthy trip state from multiple external sources.

Slide 3: Functional Requirements

Slide Title

What the System Must Do

Objective

Show the breadth of functional scope without overwhelming the audience.

On-Slide Content

- Auto-load reservations from agency systems
- Allow manual reservation entry
- Group reservations into trips
- Display active and upcoming trips
- Archive completed trips into Past Trips
- Reflect near real-time updates
- Share trip summaries to social platforms
- Highlight favored vendors
- Send critical notifications

Speaker Notes

Present these as a lifecycle rather than a list:

"A reservation enters the system either through agency integration or manual entry. It is normalized, grouped into a trip, surfaced in the active dashboard, updated as travel changes occur, and eventually archived into Past Trips once completed. Around that core lifecycle, we support social sharing, favored vendor badging, and critical notifications."

This framing makes the architecture easier to understand later because it maps directly to ingestion, grouping, presentation, notification, and archival responsibilities.

Key Message

The product behavior is a trip lifecycle pipeline.

Rubric Coverage

This slide contributes to the Functional Requirements score.

Slide 4: Architectural Drivers and Constraints

Slide Title

Why Architecture Matters Here

Objective

Explain what truly drives the design.

On-Slide Content

- Top quality goals: scalability, responsiveness, elasticity, availability
- Must support 10,000+ users now and 50,000-100,000 over time

- Peak concurrency in major travel seasons
- Must integrate through agency facade systems only
- Must coexist with existing identity and trusted infrastructure
- Must support international usage

Suggested Visual

Use the Architecture Characteristics image if helpful:

- Architecture Characteristics.png

Speaker Notes

Make it explicit that the architecture was chosen from operational pressures, not fashion:

"The dominant architectural drivers are operational. We need to absorb external updates, scale for seasonal spikes, deliver low-latency user experiences, and remain highly available for travelers who may be actively in transit."

Then explain why constraints matter:

"We do not control upstream vendor APIs directly. We integrate only through existing agency systems, and we also assume identity is delegated to the agency's current identity provider."

Key Message

Scalability and responsiveness are first-order drivers; integration constraints eliminate many simpler options.

Rubric Coverage

This slide contributes directly to the Non-Functional Requirements score.

Slide 5: Solution Strategy

Slide Title

Chosen Architecture Style

Objective

State the solution clearly and justify it.

On-Slide Content

- Event-driven microservices architecture
- Backend-for-Frontend pattern for web and mobile
- Managed cloud deployment
- Near real-time update model

Suggested Visual

Use Architecture Style.png if it already supports the narrative.

Speaker Notes

This slide should answer the question: why this style and not a modular monolith?

Use this structure:

"We chose event-driven microservices with a BFF pattern because the system has three pressures happening at the same time: high growth expectations, continuous reservation updates from external systems, and the need to deliver rich experiences across multiple client platforms."

Then explicitly address the rejected option:

"A modular monolith was considered, but it would concentrate scaling pressure, couple ingestion and presentation concerns too tightly, and make near real-time responsiveness harder to evolve independently as the system grows."

Key Message

The architecture is optimized for change, scale, and freshness, even at the cost of higher operational complexity.

Rubric Coverage

This slide is one of the primary inputs for the Solution Strategy score.

Slide 6: System Context

Slide Title

System Context: Who Talks to Road Warrior

Objective

Show boundaries and external dependencies.

Suggested Visual

Use:

- diagrams/system_context.png

On-Slide Content

- End users access via web and mobile
- Agency interface systems provide reservation data
- Social platforms receive shared trip summaries
- Identity comes from the agency identity provider
- Internal monitoring and notification systems support operations

Speaker Notes

Walk the audience from outside to inside:

"At the system boundary, Road Warrior sits between travelers and the agency's integration ecosystem. Users consume trip data through web and mobile clients. Upstream reservation data comes from agency facade systems rather than direct vendor APIs. We also depend on external social platforms for sharing, and operationally we rely on internal observability and notification capabilities."

Emphasize what is intentionally absent:

"Notice that booking systems and vendor onboarding are out of scope. This keeps the system focused on visibility and trip management."

Key Message

The context diagram proves the solution is integration-heavy and bounded by existing enterprise systems.

Rubric Coverage

This slide supports the Logical Architecture score.

Slide 7: Container Architecture

Slide Title

Container View: Major Building Blocks

Objective

Explain how responsibilities are separated inside the system.

Suggested Visual

Use:

- diagrams/container.png

On-Slide Content

- Web app and mobile app
- API gateway / entry point
- BFF layer
- Reservation ingestion service
- Trip management service
- Trip grouping service
- Notification service
- Social sharing integration
- Operational data store and messaging backbone

Speaker Notes

This is the architectural center of the presentation. Keep it crisp and capability-oriented.

Suggested walkthrough:

"The client layer is split into web and mobile experiences, each supported by Backend-for-Frontend capabilities that tailor data to the specific interaction model. Behind that, reservation ingestion handles updates from agency systems, trip management owns active and historical trip views, trip grouping is responsible for turning raw reservations into coherent itineraries, notification handles push-based updates, and social sharing isolates outbound third-party integration."

Then explain why the separation matters:

"This separation lets us scale ingestion and user-facing traffic independently, which is essential because external update bursts and end-user read traffic do not behave the same way."

Key Message

Services are split by business capability and load pattern, not by technical layers alone.

Rubric Coverage

This is the core Logical Architecture slide. Make sure it is clean, readable, and not overloaded.

Slide 8: Reservation Ingestion and Event Flow

Slide Title

How Near Real-Time Updates Enter the System

Objective

Explain the core event-driven reasoning.

On-Slide Content

- Agency systems publish reservation changes asynchronously
- Message broker decouples producers from consumers
- Reservation ingestion service validates, normalizes, and persists updates
- Downstream services react to events instead of polling synchronously

Speaker Notes

This is where you justify ADR-2 in practical terms:

"Reservation updates are bursty and externally controlled. An event-driven ingestion model protects the platform from upstream variability by buffering spikes through the broker and processing updates asynchronously. That gives us resilience, loose coupling, and a cleaner path to near real-time freshness than tightly coupled request-response integration."

Then mention the engineering consequences:

"The trade-off is eventual consistency and the need for idempotent consumers, schema validation, retry logic, and dead-letter handling."

Key Message

Events are the mechanism that converts unreliable external update timing into a scalable internal processing model.

Rubric Coverage

This slide supports both Logical Architecture and Solution Strategy scoring.

Slide 9: Trip Grouping Logic and User Experience

Slide Title

From Raw Reservations to Trips

Objective

Show how the product becomes useful to travelers.

On-Slide Content

- Group by date proximity
- Group by geographic continuity
- Use logical sequencing across flight, hotel, and car
- Allow user override to merge or split trips
- Move completed trips automatically to Past Trips

Speaker Notes

This is a high-value differentiator slide because it connects architecture to user experience.

"Users do not think in isolated reservations. They think in trips. That means the architecture needs a dedicated grouping capability rather than burying this logic inside a generic CRUD service. The grouping service uses temporal proximity, location continuity, and sequence patterns to infer trips automatically, while still allowing user correction when real-world travel does not match the heuristic."

Then connect to archival:

"Once the trip end date has passed, the system transitions that trip out of the active dashboard and into Past Trips automatically, preserving historical access without cluttering the primary experience."

Key Message

Trip grouping is a domain capability, not a UI convenience.

Rubric Coverage

This slide strengthens both Logical Architecture and requirement traceability.

Slide 10: Rich User Experience Across Platforms

Slide Title

Why We Chose Backend-for-Frontend

Objective

Justify the BFF pattern directly against the problem statement.

On-Slide Content

- Different payload and interaction needs for web and mobile
- Faster page/app rendering through tailored APIs
- Cleaner support for notifications and live updates
- Reduced over-fetching and client-side orchestration

Speaker Notes

This slide should be very direct:

"The problem statement explicitly asks for the richest user interface possible across deployment platforms. That requirement is not just visual. It means the backend should optimize for device-specific data access patterns. A BFF lets us shape payloads and workflows for web and mobile independently without forcing every client through a one-size-fits-all API."

Then note operational benefit:

"It also reduces frontend complexity by moving client orchestration into backend components that we can version, monitor, and evolve more safely."

Key Message

BFF is the bridge between a complex backend and a high-quality multi-platform user experience.

Rubric Coverage

This slide directly supports Solution Strategy by mapping the rich UI requirement to the architecture.

Slide 11: Runtime View

Slide Title

Runtime Scenario: Reservation Update to Traveler Dashboard

Objective

Show the end-to-end dynamic behavior of the system.

Suggested Visual

Use:

- diagrams/runtime_view.png

On-Slide Content

- External update arrives
- Event is processed and stored
- Trip grouping re-evaluates impacted trip
- Notification is emitted if needed
- Dashboard refreshes through live update channel

Speaker Notes

Narrate one concrete scenario, for example a flight delay:

"A flight delay is published by the agency interface system. The event enters the broker, the ingestion service validates and normalizes it, the trip state is updated, the affected trip is re-evaluated, and the notification service pushes a live update to the traveler. The result is that the user sees the changed trip state quickly, without manually refreshing or waiting for a periodic sync cycle."

This slide is important because it proves the architecture is behaviorally coherent, not just structurally decomposed.

Key Message

The runtime view demonstrates how the chosen architecture delivers freshness and responsiveness together.

Rubric Coverage

This slide supports Logical Architecture clarity and makes the system easier for reviewers to follow.

Slide 12: Data and Persistence Strategy

Slide Title

Why Azure SQL Is the Operational Database

Objective

Explain the database decision in business terms.

On-Slide Content

- Structured domain: users, reservations, trips
- Strong consistency for updates and manual entry
- Efficient relational queries for trip views
- Managed PaaS operations

Speaker Notes

This slide should connect ADR-5 to the actual domain model:

"The data model is relational by nature. Users own trips, trips contain reservations, and those reservations have strong relationships and lifecycle state. Azure SQL gives us transactional integrity, mature querying, and managed operations, which matches the current scope better than a schema-flexible NoSQL model."

Then be candid about the trade-off:

"The downside is that very large-scale horizontal partitioning is more complex than in some NoSQL systems, but for the expected user growth and reservation volume, Azure SQL remains a pragmatic and defensible choice."

Key Message

Relational consistency and queryability matter more here than maximum schema flexibility.

Rubric Coverage

This slide supports Solution Strategy by tying a technology choice to domain and consistency needs.

Slide 13: Deployment and Operations

Slide Title

Cloud-Native Deployment for Scale and Resilience

Objective

Show how the solution meets scalability, elasticity, and availability goals operationally.

Suggested Visual

Use:

- diagrams/deployment.png

On-Slide Content

- Managed compute platform
- Managed messaging
- Managed database
- Auto-scaling services
- Observability: logs, metrics, traces
- Secure integration within agency trust boundary

Speaker Notes

Explain that deployment is part of the architecture, not an afterthought:

"Because the main quality goals are operational, deployment decisions matter as much as service boundaries. Managed cloud services reduce undifferentiated operational burden, improve elasticity, and

allow the team to focus on business capabilities rather than infrastructure maintenance."

Then address reliability:

"This also supports high availability and better disaster recovery patterns, while observability gives us the ability to detect ingestion lag, message failures, and user-facing latency before they become customer-visible incidents."

Key Message

Managed cloud services make the proposed architecture sustainable for a growing platform team.

Rubric Coverage

This is the main Physical Architecture slide. It must clearly show what runs where and why.

Slide 14: Key ADRs and Trade-Offs

Slide Title

Architecture Decisions We Made Deliberately

Objective

Summarize the most important decisions and show disciplined reasoning.

On-Slide Content

- ADR-1: Microservices architecture
- ADR-2: Asynchronous event-driven updates
- ADR-3: WebSockets/SSE for live client updates
- ADR-4: Managed cloud-native deployment
- ADR-5: Azure SQL as primary operational store
- ADR-6: Automatic trip grouping with user override

Speaker Notes

Do not read ADR titles mechanically. Use them as a compressed reasoning summary:

"These ADRs show that our architecture is not a collection of disconnected choices. Each decision supports a specific pressure: scale, freshness, responsiveness, operational simplicity, data integrity, or domain usability."

Then mention the accepted costs:

"What we accepted in return was greater distributed-system complexity: eventual consistency, more operational moving parts, and the need for stronger observability and failure handling."

Key Message

The architecture is a set of explicit trade-offs, not a generic microservices template.

Rubric Coverage

This is the main ADR slide. Keep each decision concise and defensible.

Slide 15: Risks and Mitigations

Slide Title

What Could Go Wrong and How We Reduce the Risk

Objective

Demonstrate architectural maturity.

On-Slide Content

- Stale or inconsistent reservation state
- Integration unreliability from agency systems
- Peak-season traffic surges
- Real-time connection management complexity
- Security and privacy exposure

Speaker Notes

Use a calm, engineering-focused tone:

"The biggest failure mode is loss of trust in trip data. If updates lag or grouping is wrong, the product loses credibility quickly. That is why we emphasize idempotent event processing, schema validation, reconciliation, dead-letter handling, and robust observability."

Then address security:

"Travel data is sensitive. Even in a trusted enterprise boundary, we still require strong authentication, authorization, encryption in transit and at rest, and least-privilege service access."

Key Message

The design is realistic because it acknowledges the hard parts and includes mitigations, not because it assumes ideal conditions.

Slide 16: Closing Summary

Slide Title

Why This Architecture Fits Road Warrior

Objective

End with a concise, defensible conclusion.

On-Slide Content

- Meets the core functional scope
- Scales for projected user growth
- Supports near real-time updates
- Delivers rich web and mobile experiences
- Keeps architecture adaptable for future evolution

Speaker Notes

Suggested closing:

"Road Warrior needs to consolidate fragmented travel data, keep it current, and present it in a way that feels coherent to travelers across platforms. Our event-driven microservices architecture with BFF addresses those needs directly. It is not the simplest architecture possible, but it is the most appropriate for the system's integration complexity, growth expectations, and user experience goals."

End with:

"In short, we optimized for trustworthy trip state, operational scalability, and product responsiveness."

Key Message

The proposed architecture is justified by the problem, not just by architectural preference.

Optional Backup Slides

Use these only if there is time or if reviewers ask deeper questions.

Backup Slide A: Why Not a Modular Monolith?

Use this slide if challenged on complexity.

Key points:

- A modular monolith could reduce operational complexity initially.
- It was rejected because ingestion bursts, real-time updates, and multi-platform API needs create uneven scaling pressures.
- Independent scaling and separation of failure domains were judged more valuable than reduced deployment simplicity.

Backup Slide B: International Considerations

Key points:

- Initial assumption is English and UTC-centric handling.
- Future evolution should include full locale, timezone, and regional compliance support.
- Cloud-native deployment supports geographic expansion more easily than fixed on-prem assumptions.

Backup Slide C: Favored Vendors and Social Sharing

Key points:

- Favored vendor status is treated as upstream metadata, not a business rule engine in Road Warrior.
 - Social sharing is intentionally lightweight and text-based.
 - Both features are isolated from the core trip lifecycle so they do not destabilize primary trip management functions.
-

Slide Design Guidance

Use the following rules when building the actual deck:

- Keep each slide to 3-5 bullets maximum.
- Put detail in speaker notes, not on the slide body.
- Use the four existing diagrams as anchor slides rather than redrawing new architecture visuals unless necessary.
- When presenting trade-offs, explicitly name both the benefit and the cost.
- Avoid turning the presentation into a technology catalog. Keep returning to the business problem and quality goals.

Recommended diagram usage from the repo:

- system context slide: `diagrams/system_context.png`
- container slide: `diagrams/container.png`
- runtime slide: `diagrams/runtime_view.png`
- deployment slide: `diagrams/deployment.png`

Optional supporting visuals from repo root:

- `Architecture Characteristics.png`
 - `Architecture Style.png`
-

Recommended Opening and Closing Script

Opening Script

"Road Warrior is a trip management dashboard for travelers who need one coherent, up-to-date view of their reservations across flights, hotels, and car rentals. The architectural challenge is not just displaying data. It is integrating fragmented upstream systems, keeping trip state fresh, and delivering a responsive experience across web and mobile at scale."

Closing Script

"Our final design uses event-driven microservices, a Backend-for-Frontend layer, and managed cloud services to meet the primary demands of scale, responsiveness, and availability. The result is an architecture built to handle changing travel data, growing user demand, and the operational realities of enterprise integration."

Final Presenter Checklist

- Verify the deck visibly satisfies all five rubric categories.
- Confirm all four architecture diagrams are readable in the deck.
- Keep the story aligned to requirements, drivers, decisions, and trade-offs.
- Be ready to explain why a monolith was not chosen.
- Be ready to explain how eventual consistency is controlled.
- Be ready to explain why trip grouping is a dedicated capability.
- Keep repeating the product scope: trip management, not booking.